DATE: 10-10-2026

CO-1 ASSESSMENT TOOL-1

(APPLICATION BASED QUESTIONS)

COURSE CODE:DSA0308

COURSE NAME: NATURAL LANGUAGE PROCESSING

NAME: P.GAYATHRI **REG.NO:192425277**

Q-1

Section 1: Regular Expressions – Resume Information Extraction

1. A multinational recruitment company receives thousands of resumes every day in text format. The HR department requires an automated system to extract candidate information for shortlisting.

ANSWER:

- The re module is used in Python to perform Regular Expression operations and extract information from resume text.
- The candidate's name is extracted by identifying the name field in the resume.
- Email addresses are identified by checking the standard email format, which contains a username, the @ symbol, a domain name, and an extension.
- Mobile numbers are extracted by checking the required mobile number format, such as a 10-digit Indian mobile number.

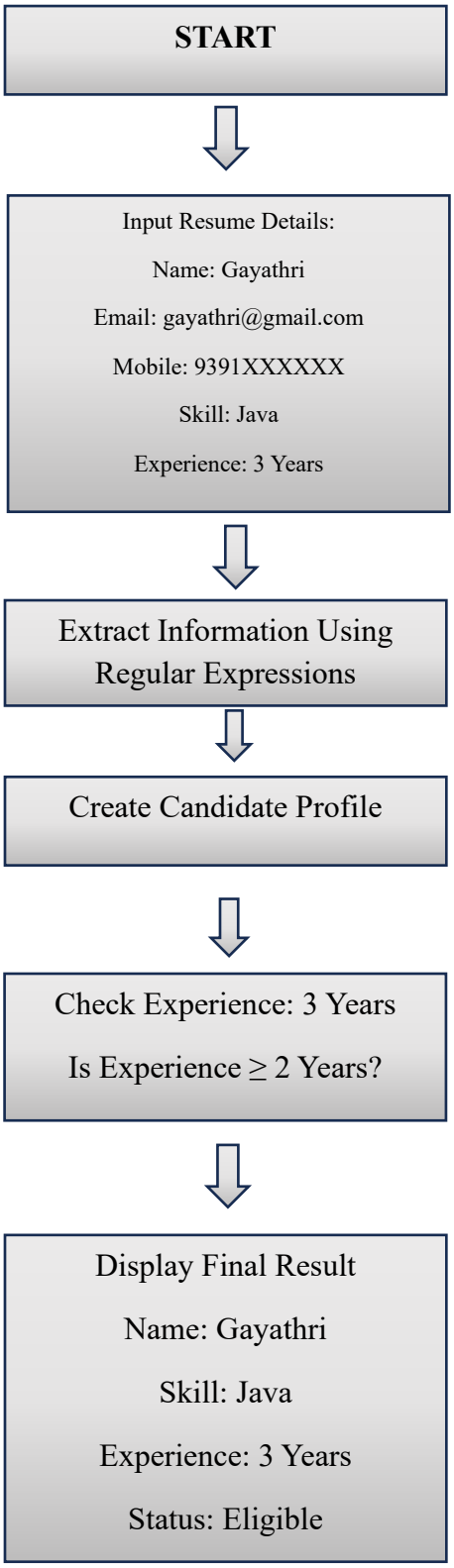
- Technical skills such as Python, Java, SQL, Machine Learning, and NLP are detected by matching these keywords with the resume text.
- The re.search() function is used to find specific information in the resume.
- The re.ignorecase flag helps detect technical skills even when they are written in uppercase or lowercase letters.
- Years of experience are extracted by identifying numerical values followed by words such as years or years of experience.
- The extracted experience value is converted into a number so that it can be compared with the minimum eligibility requirement.
- A structured candidate profile is created containing the candidate's name, email address, mobile number, technical skills, and years of experience.
- The extracted details are organized into a dictionary or another structured format for easy access and display.
- The eligibility condition checks whether the candidate has a minimum of two years of experience.
- The system also checks whether the candidate possesses Python skills.
- Candidates who have at least two years of experience and Python skills are marked as eligible. Candidates who do not satisfy either requirement are marked as not eligible.
- Finally, the system displays the candidate's complete profile and eligibility status, helping the HR department reduce manual work and shortlist suitable candidates efficiently.

EX: NAME: P.GAYATHRI

PNONE :9391XXXXXX

SKILL: JAVA

FLOW CHAT


START
Input Resume Details:
Name: Gayathri
Email: gayathri@gmail.com
Mobile: 9391XXXXXX
Skill: Java
Experience: 3 Years
Extract Information Using Regular Expressions
Create Candidate Profile
Check Experience: 3 Years
Is Experience ≥ 2 Years?
Display Final Result
Name: Gayathri
Skill: Java
Experience: 3 Years
Status: Eligible

CODE:

```
import re
resume = """Name: Gayathri
Email: gayathri@gmail.com
Mobile: 9391234567
Skills: Python, Java, SQL, NLP
Experience: 3 years"""
name = re.search(r"Name:\s*(.*)", resume).group(1)
email = re.search(r"\S+@\S+\.\S+", resume).group()
mobile = re.search(r"\d{10}", resume).group()
skills = re.findall(r"Python|Java|SQL|Machine Learning|NLP", resume)
exp = int(re.search(r"(\d+)\s*years", resume).group(1))
print("Name:", name)
print("Email:", email)
print("Mobile:", mobile)
print("Skills:", skills)
print("Experience:", exp, "years")
if exp >= 2 and "Python" in skills:
    print("Status: Eligible")
else: print("Status: Not Eligible")
```

OUTPUT:

```
Name: Gayathri
Email: gayathri@gmail.com
Mobile: 9391234567
Skills: ['Python', 'Java', 'SQL', 'NLP']
Experience: 3 years
Status: Eligible
```

Q-2

Section 2: Regular Expressions – Product Search System

2. An e-commerce company plans to enhance its product search engine to improve customer experience through flexible keyword matching.

ANSWER:

- First, a list of product names is collected from the e-commerce website.
- The Python re module is used to perform product searches using Regular Expressions.
- Exact Keyword Search: Searches for products containing an exact keyword, such as “Laptop”.
- Prefix-Based Search: Finds products whose names begin with a particular keyword. For example, searching “Sam” displays “Samsung Mobile” and “Samsung TV”.
- Suffix-Based Search: Finds products whose names end with a particular keyword. For example, searching “Phone” displays “Smart Phone” and “iPhone”.
- Partial Keyword Search: Searches for products containing a part of a word. For example, searching “Book” displays “Notebook”, “Book Shelf” and “Books”.
- Case-Insensitive Search: Matches keywords without considering uppercase or lowercase letters. For example, “laptop”, “Laptop” and “LAPTOP” produce the same matching results.
- The re.search() function is used to find keywords within product names.
- Regular Expression patterns are prepared according to the selected search type.

- The system compares the search keyword with each product name in the product list.
- All matching product names are collected and displayed to the customer.
- If no products match the keyword, the system displays a message stating that no matching products were found.
- The total number of matching products is calculated for each search operation.
- A search report is generated containing the search keyword, search type, matching product names and total number of matches.
- Finally, the system displays the search results and report, helping customers find products easily and improving the overall shopping experience.

CODE:

```
import re

products = ["Samsung Mobile", "Laptop", "Smart Phone",

        "Notebook", "Samsung TV", "iPhone"]

keyword = input("Enter search keyword: ")

matches = [p for p in products

        if re.search(keyword, p, re.IGNORECASE)]

print("Matching Products:", matches)

print("Total Matches:", len(matches))
```

OUTPUT:

```
Enter search keyword: Samsung

Matching Products: ['Samsung Mobile', 'Samsung TV']

Total Matches: 2
```

Q-3

Section 3: Regular Expressions – University Registration System

A university is developing an automated online registration portal to verify student information before course enrollment.

ANSWER:

- First, the student enters the register number, institutional email address, course code, semester and mobile number.
- The Python re module is used to validate the student information using Regular Expressions.
- Register Number Validation: Checks whether the student register number follows the university's required format.
- Institutional Email Validation: Verifies that the email address follows the correct format and uses the university's institutional domain.
- Course Code Validation: Checks whether the entered course code matches the prescribed format, such as CS101.
- Semester Validation: Checks whether the entered semester is within the university's permitted semester range.
- Mobile Number Validation: Verifies that the mobile number contains the required number of digits and follows the expected format.
- The re.match() or re.fullmatch() function is used to check whether the entered information matches the required pattern.
- Each field is validated separately to identify correct and incorrect information.
- If a field is valid, the system displays a message such as “Valid Register Number” or “Valid Email Address”.

- If a field is incorrect, the system displays an appropriate error message asking the student to enter the information in the correct format.
- After validating all fields, the system checks whether every required detail is valid.
- If all fields are valid, the registration status is displayed as “Registration Successful”.
- If any field is invalid, the registration status is displayed as “Registration Failed – Please Correct the Invalid Details”.
- Finally, the system generates a registration report containing the student's details, individual validation results and the final registration status.

CODE:

```
import re

reg = input("Register Number: ")

email = input("Institutional Email: ")

course = input("Course Code: ")

sem = input("Semester: ")

mobile = input("Mobile Number: ")

checks = {

   "Register Number": bool(re.fullmatch(r"\d{12}", reg)),

   "Email": bool(re.fullmatch(r"[\w.]+@university\.edu", email)),

   "Course Code": bool(re.fullmatch(r"[A-Z]{2}\d{3}", course)),

   "Semester": bool(re.fullmatch(r"[1-8]", sem)),

   "Mobile Number": bool(re.fullmatch(r"\d{10}", mobile))

}
```

```
for field, valid in checks.items():
    print(field, "Valid" if valid else "Invalid")
print("Registration Successful" if all(checks.values())
      else "Registration Failed")
```

OUTPUT:

Register Number: 123456789012

Institutional Email: gayathri@university.edu

Course Code: CS101

Semester: 3

Mobile Number: 9876543210

Register Number Valid

Email Valid

Course Code Valid

Semester Valid

Mobile Number Valid

Registration Successful